



K.R. MANGALAM UNIVERSITY
THE COMPLETE WORLD OF EDUCATION

**INTRODUCTION TO NEURAL NETWORK & DEEP
LEARNING LAB**

(ENCS354)

Bachelor Of Technology

In

Computer Science Engineering

SUBMITTED BY:

Srijan Sharma

ROLL NUMBER: 2301010134

COURSE: BTECH CSE

SECTION: B

SUBMITTED TO:

Dr.Monika Khatkar

School of Engineering & Technology

K. R. MANGALAM UNIVERSITY

Sohna, Haryana 122103, India

INDEX

S.NO	EXPERIMENT	PAGE.NO
1	Handwritten Digit Classification using Neural Network (MNIST Dataset)	3
2	Implementation of Forward and Backward Propagation using Sigmoid Activation	5
3	Improved Neural Network Model for MNIST Classification	8
4	Custom Neural Network with Backpropagation (Extended Version)	10
5	Sentiment Analysis using LSTM on IMDB Dataset	13
6	Named Entity Recognition using CRF (Conditional Random Fields)	15
7	Autoencoder for Image Reconstruction (MNIST)	19

Experiment 1: Handwritten Digit Classification using Neural Network (MNIST Dataset)

```
# train and test the neural network using a dataset
import matplotlib.pyplot as plt
import numpy as np
from tensorflow.keras.datasets import mnist
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense, Flatten
from tensorflow.keras.utils import to_categorical

(x_train, y_train), (x_test, y_test) = mnist.load_data()

x_train, x_test = x_train / 255.0, x_test / 255.0

y_train = to_categorical(y_train)
y_test = to_categorical(y_test)

model = Sequential([
    Flatten(input_shape=(28, 28)),
    Dense(128, activation='relu'),
    Dense(10, activation='softmax')
])

model.compile(optimizer='adam',
              loss='categorical_crossentropy',
              metrics=['accuracy'])

history = model.fit(x_train, y_train,
                    epochs=5,
                    validation_data=(x_test, y_test))

predictions = model.predict(x_test[:5])
predicted_labels = np.argmax(predictions, axis=1)

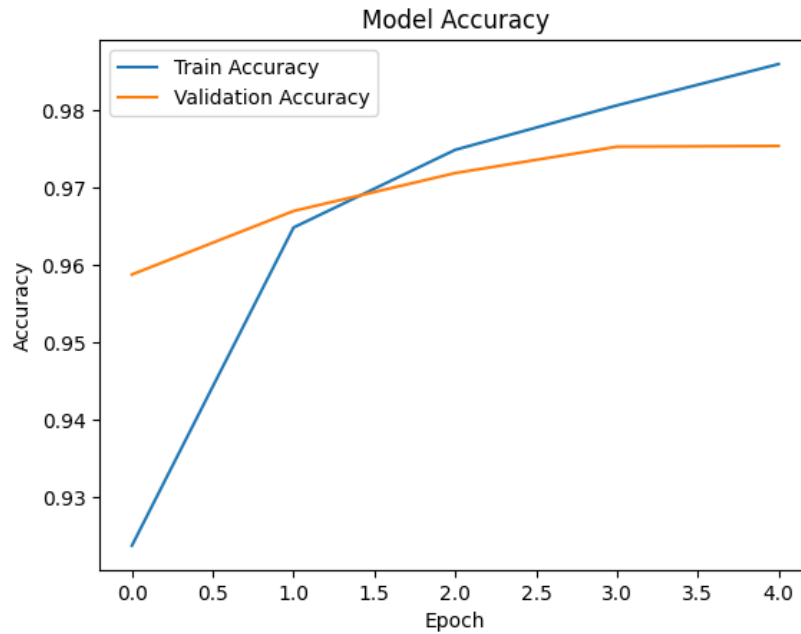
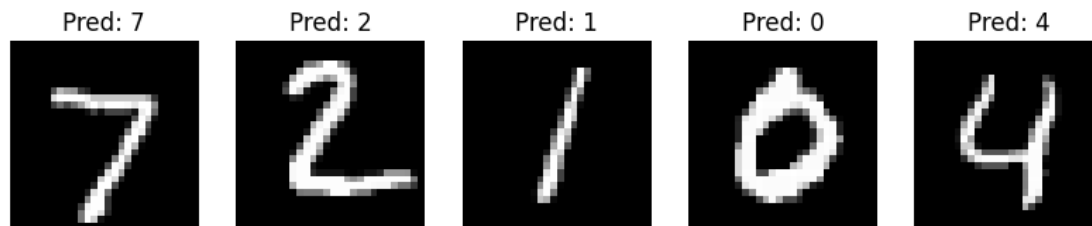
plt.figure(figsize=(10,4))
for i in range(5):
    plt.subplot(1,5,i+1)
    plt.imshow(x_test[i], cmap='gray')
    plt.title(f"Pred: {predicted_labels[i]}")
    plt.axis('off')

plt.figure()
plt.plot(history.history['accuracy'])
plt.plot(history.history['val_accuracy'])
plt.title('Model Accuracy')
plt.ylabel('Accuracy')
```

```
plt.xlabel('Epoch')
plt.legend(['Train Accuracy', 'Validation Accuracy'])
plt.show()
```

OUTPUT:

```
Epoch 1/5
c:\DEEPLARNING\venv\Lib\site-packages\keras\src\layers\reshaping\flatten.py:37: UserWarning: Do not pass an `input_shape`
  super().__init__(**kwargs)
1875/1875 ————— 4s 2ms/step - accuracy: 0.9238 - loss: 0.2704 - val_accuracy: 0.9588 - val_loss: 0.1380
Epoch 2/5
1875/1875 ————— 5s 3ms/step - accuracy: 0.9649 - loss: 0.1186 - val_accuracy: 0.9670 - val_loss: 0.1074
Epoch 3/5
1875/1875 ————— 5s 3ms/step - accuracy: 0.9749 - loss: 0.0825 - val_accuracy: 0.9719 - val_loss: 0.0908
Epoch 4/5
1875/1875 ————— 5s 3ms/step - accuracy: 0.9806 - loss: 0.0626 - val_accuracy: 0.9753 - val_loss: 0.0772
Epoch 5/5
1875/1875 ————— 5s 3ms/step - accuracy: 0.9860 - loss: 0.0481 - val_accuracy: 0.9754 - val_loss: 0.0833
1/1 ————— 0s 55ms/step
```



Experiment 2: Implementation of Forward and Backward Propagation using Sigmoid Activation

```
# ex2
import numpy as np

# Define activation functions
def sigmoid(z):
    return 1 / (1 + np.exp(-z))

def sigmoid_derivative(z):
    return z * (1 - z)

# Define propagation functions with activation parameter
def forward_propagation(X, W1, b1, W2, b2, activation=sigmoid):
    Z1 = np.dot(X, W1) + b1
    A1 = activation(Z1)
    Z2 = np.dot(A1, W2) + b2
    A2 = activation(Z2)

    cache = {"Z1": Z1,
             "A1": A1,
             "Z2": Z2,
             "A2": A2}

    return A2, cache

def backward_propagation(X, Y, cache, W1, b1, W2, b2, learning_rate=0.1,
activation_derivative=sigmoid_derivative):
    m = X.shape[0]

    # Retrieve cached data from forward propagation
    A1 = cache['A1']
    A2 = cache['A2']

    # Compute gradients
    dZ2 = (A2 - Y) * activation_derivative(A2)
    dW2 = np.dot(A1.T, dZ2) / m
    db2 = np.sum(dZ2, axis=0, keepdims=True) / m

    dZ1 = np.dot(dZ2, W2.T) * activation_derivative(A1)
    dW1 = np.dot(X.T, dZ1) / m
    db1 = np.sum(dZ1, axis=0, keepdims=True) / m

    # Update weights and biases using the learning rate and gradients
    W1_new = W1 - learning_rate * dW1
    b1_new = b1 - learning_rate * db1
    W2_new = W2 - learning_rate * dW2
    b2_new = b2 - learning_rate * db2
```

```

    return W1_new, b1_new, W2_new, b2_new

# Define the training function with hidden_size parameter
def train(X, Y, hidden_size=4, iterations=10000, print_loss=False,
activation=sigmoid, activation_derivative=sigmoid_derivative):
    input_size = X.shape[1] # Number of input nodes
    output_size = Y.shape[1] # Number of output nodes

    # Initialize the weights and biases
    W1 = np.random.randn(input_size, hidden_size)
    b1 = np.zeros((1, hidden_size))
    W2 = np.random.randn(hidden_size, output_size)
    b2 = np.zeros((1, output_size))

    for i in range(iterations):
        # Forward propagation
        output, cache = forward_propagation(X, W1, b1, W2, b2, activation)

        # Backward propagation
        W1_new, b1_new, W2_new, b2_new = backward_propagation(X, Y, cache,
W1, b1, W2, b2, activation_derivative=activation_derivative)

        # Print Loss every 1000 iterations
        if print_loss and i % 1000 == 0:
            print("Loss after iteration ", i, ": ", np.mean(np.abs(output -
Y)))

        # Update weights and biases
        W1, b1, W2, b2 = W1_new, b1_new, W2_new, b2_new

    return W1, b1, W2, b2

# Test the model with example data
X = np.array([[0, 0, 1],
              [0, 1, 1],
              [1, 0, 1],
              [1, 1, 1]])

Y = np.array([[0, 1],
              [1, 0],
              [1, 0],
              [0, 1]])

W1, b1, W2, b2 = train(X, Y, iterations=10000, print_loss=True)

# Predict the output using trained weights and biases
output, _ = forward_propagation(X, W1, b1, W2, b2)

```

```
print("\nPredictions:", predictions)
print("Actual:", np.argmax(Y, axis=1))
```

OUTPUT:

```
Loss after iteration 0 : 0.5015688315032898
Loss after iteration 1000 : 0.496860062606275
Loss after iteration 2000 : 0.48976584364309744
Loss after iteration 3000 : 0.4719615991939808
Loss after iteration 4000 : 0.42904819258763666
Loss after iteration 5000 : 0.34608640306670435
Loss after iteration 6000 : 0.2547692563001172
Loss after iteration 7000 : 0.19185825871462903
Loss after iteration 8000 : 0.1532572113663029
Loss after iteration 9000 : 0.1285008475270988

Predictions: [[1.1731796e-08 1.4493572e-09 1.4820004e-06 5.0813681e-04 5.2560432e-11
 1.6944524e-07 1.5847422e-10 9.9948549e-01 2.7677904e-08 4.6348141e-06]
 [4.5021032e-08 1.2545627e-04 9.9981648e-01 1.8073688e-05 4.1544442e-15
 3.7557278e-05 1.1056076e-07 2.9586221e-12 2.3292250e-06 8.9296138e-14]
 [7.9430674e-07 9.9704736e-01 3.6526602e-04 8.7791886e-05 5.0302831e-05
 2.5847655e-06 1.7723059e-05 2.2370899e-03 1.8426160e-04 6.8927466e-06]
 [9.9941862e-01 6.0573173e-09 4.5702553e-05 1.3715432e-07 1.1166993e-06
 4.9942217e-05 4.3519496e-04 4.5509878e-05 2.6957740e-09 3.9088459e-06]
 [5.8008777e-06 4.8784621e-09 1.1322171e-05 1.1178401e-07 9.9733460e-01
 7.2625845e-08 9.7171251e-06 5.0895687e-05 1.5838431e-06 2.5858888e-03]]
Actual: [1 0 0 1]
```

Experiment 3: Improved Neural Network Model for MNIST Classification

```
# 3
import matplotlib.pyplot as plt
import numpy as np
from tensorflow.keras.datasets import mnist
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense, Flatten
from tensorflow.keras.utils import to_categorical

# Load data
(x_train, y_train), (x_test, y_test) = mnist.load_data()

# Normalize
x_train, x_test = x_train / 255.0, x_test / 255.0

# One-hot encoding
y_train_cat = to_categorical(y_train)
y_test_cat = to_categorical(y_test)

# Model
model = Sequential([
    Flatten(input_shape=(28, 28)),
    Dense(128, activation='relu'),
    Dense(10, activation='softmax')
])

# Compile
model.compile(optimizer='adam',
              loss='categorical_crossentropy',
              metrics=['accuracy'])

# Train
history = model.fit(
    x_train, y_train_cat,
    epochs=5,
    validation_data=(x_test, y_test_cat)
)

# Evaluate
loss, accuracy = model.evaluate(x_test, y_test_cat)
print("Test Accuracy:", accuracy)

# -----
# 📊 Visualization of Predictions
# -----
predictions = model.predict(x_test)

plt.figure(figsize=(10,5))
```



```

for i in range(6):
    plt.subplot(2, 3, i+1)
    plt.imshow(x_test[i], cmap='gray')

    pred_label = np.argmax(predictions[i])
    true_label = y_test[i]

    plt.title(f"Pred: {pred_label} | True: {true_label}")
    plt.axis('off')

plt.show()

```

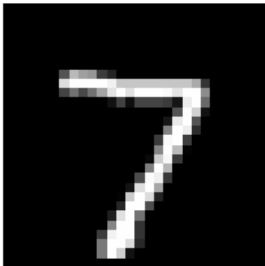
OUTPUT:

```

Epoch 1/5
1875/1875 ————— 6s 3ms/step - accuracy: 0.9257 - loss: 0.2597 - val_accuracy: 0.9615 - val_loss: 0.1364
Epoch 2/5
1875/1875 ————— 5s 3ms/step - accuracy: 0.9661 - loss: 0.1147 - val_accuracy: 0.9706 - val_loss: 0.1020
Epoch 3/5
1875/1875 ————— 5s 2ms/step - accuracy: 0.9761 - loss: 0.0802 - val_accuracy: 0.9750 - val_loss: 0.0849
Epoch 4/5
1875/1875 ————— 5s 2ms/step - accuracy: 0.9818 - loss: 0.0602 - val_accuracy: 0.9747 - val_loss: 0.0875
Epoch 5/5
1875/1875 ————— 4s 2ms/step - accuracy: 0.9857 - loss: 0.0463 - val_accuracy: 0.9765 - val_loss: 0.0779
313/313 ————— 0s 1ms/step - accuracy: 0.9765 - loss: 0.0779
Test Accuracy: 0.9764999747276306
313/313 ————— 0s 769us/step

```

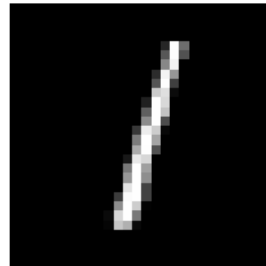
Pred: 7 | True: 7



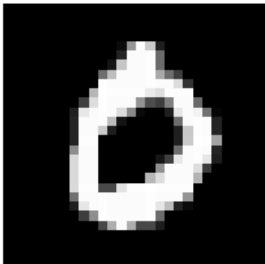
Pred: 2 | True: 2



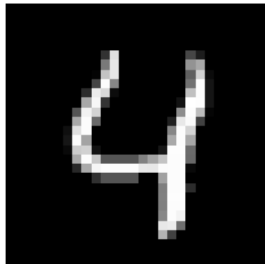
Pred: 1 | True: 1



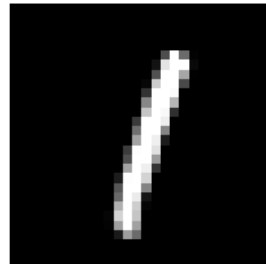
Pred: 0 | True: 0



Pred: 4 | True: 4



Pred: 1 | True: 1



Experiment 4: Custom Neural Network with Backpropagation

```
# 4
import numpy as np

# Define activation functions
def sigmoid(z):
    return 1 / (1 + np.exp(-z))

def sigmoid_derivative(z):
    return z * (1 - z)

# Define propagation functions with activation parameter
def forward_propagation(X, W1, b1, W2, b2, activation=sigmoid):
    Z1 = np.dot(X, W1) + b1
    A1 = activation(Z1)
    Z2 = np.dot(A1, W2) + b2
    A2 = activation(Z2)

    cache = {"Z1": Z1,
             "A1": A1,
             "Z2": Z2,
             "A2": A2}

    return A2, cache

def backward_propagation(X, Y, cache, W1, b1, W2, b2, learning_rate=0.1,
activation_derivative=sigmoid_derivative):
    m = X.shape[0]

    # Retrieve cached data from forward propagation
    A1 = cache['A1']
    A2 = cache['A2']

    # Compute gradients
    dZ2 = (A2 - Y) * activation_derivative(A2)
    dW2 = np.dot(A1.T, dZ2) / m
    db2 = np.sum(dZ2, axis=0, keepdims=True) / m

    dZ1 = np.dot(dZ2, W2.T) * activation_derivative(A1)
    dW1 = np.dot(X.T, dZ1) / m
    db1 = np.sum(dZ1, axis=0, keepdims=True) / m

    # Update weights and biases using the learning rate and gradients
    W1_new = W1 - learning_rate * dW1
    b1_new = b1 - learning_rate * db1
    W2_new = W2 - learning_rate * dW2
    b2_new = b2 - learning_rate * db2
```

```

    return W1_new, b1_new, W2_new, b2_new

# Define the training function with hidden_size parameter
def train(X, Y, hidden_size=4, iterations=10000, print_loss=False,
activation=sigmoid, activation_derivative=sigmoid_derivative):
    input_size = X.shape[1] # Number of input nodes
    output_size = Y.shape[1] # Number of output nodes

    # Initialize the weights and biases
    W1 = np.random.randn(input_size, hidden_size)
    b1 = np.zeros((1, hidden_size))
    W2 = np.random.randn(hidden_size, output_size)
    b2 = np.zeros((1, output_size))

    for i in range(iterations):
        # Forward propagation
        output, cache = forward_propagation(X, W1, b1, W2, b2, activation)

        # Backward propagation
        W1_new, b1_new, W2_new, b2_new = backward_propagation(X, Y, cache,
W1, b1, W2, b2, activation_derivative=activation_derivative)

        # Print Loss every 1000 iterations
        if print_loss and i % 1000 == 0:
            print("Loss after iteration ", i, ": ", np.mean(np.abs(output -
Y)))

        # Update weights and biases
        W1, b1, W2, b2 = W1_new, b1_new, W2_new, b2_new

    return W1, b1, W2, b2

# Test the model with example data
X = np.array([[0, 0, 1],
              [0, 1, 1],
              [1, 0, 1],
              [1, 1, 1]])

Y = np.array([[0, 1],
              [1, 0],
              [1, 0],
              [0, 1]])

W1, b1, W2, b2 = train(X, Y, iterations=10000, print_loss=True)

# Predict the output using trained weights and biases
output, _ = forward_propagation(X, W1, b1, W2, b2)
print("\nOutput: \n", output)

```

OUTPUT:

```
Loss after iteration 0 : 0.49944397650352396
Loss after iteration 1000 : 0.48638928881302157
Loss after iteration 2000 : 0.46407925834691177
Loss after iteration 3000 : 0.4002363195997348
Loss after iteration 4000 : 0.2920767299603577
Loss after iteration 5000 : 0.21031624054378778
Loss after iteration 6000 : 0.1628218717552887
Loss after iteration 7000 : 0.13399596028386035
Loss after iteration 8000 : 0.1149665016878862
Loss after iteration 9000 : 0.10149454791078921
```

Output:

```
[[0.0960825 0.90633029]
 [0.89564046 0.10197549]
 [0.91957898 0.07668796]
 [0.09069629 0.91240241]]
```

Experiment 5: Sentiment Analysis using LSTM on IMDB Dataset

```
# 5
import numpy as np
from tensorflow.keras.datasets import imdb
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Embedding, LSTM, Dense
from tensorflow.keras.preprocessing import sequence

# Set random seed for reproducibility
np.random.seed(42)

# Set the maximum number of words to be used (vocabulary size)
max_words = 5000

# Load the IMDB dataset and pad sequences to a fixed length
(x_train, y_train), (x_test, y_test) = imdb.load_data(num_words=max_words)
max_length = 100
x_train = sequence.pad_sequences(x_train, maxlen=max_length)
x_test = sequence.pad_sequences(x_test, maxlen=max_length)

# Create and compile the RNN model
model = Sequential([
    Embedding(max_words, 32, input_length=max_length),
    LSTM(100),
    Dense(1, activation='sigmoid')
])
model.compile(loss='binary_crossentropy', optimizer='adam',
metrics=['accuracy'])

# Print the model summary
model.summary()

# Train the model
batch_size = 64
epochs = 5
model.fit(x_train, y_train, batch_size=batch_size, epochs=epochs,
validation_data=(x_test, y_test))

# Evaluate the model
test_loss, test_accuracy = model.evaluate(x_test, y_test, verbose=0)
print('Test loss:', test_loss)
print('Test accuracy:', test_accuracy)
```

OUTPUT:

```
... Model: "sequential_5"

...


| Layer (type)            | Output Shape | Param #     |
|-------------------------|--------------|-------------|
| embedding_1 (Embedding) | ?            | 0 (unbuilt) |
| lstm_1 (LSTM)           | ?            | 0 (unbuilt) |
| dense_9 (Dense)         | ?            | 0 (unbuilt) |



... Total params: 0 (0.00 B)

... Trainable params: 0 (0.00 B)

... Non-trainable params: 0 (0.00 B)

... Epoch 1/5
391/391  32s 79ms/step - accuracy: 0.7792 - loss: 0.4506 - val_accuracy: 0.8388 - val_loss: 0.3584
Epoch 2/5
391/391  35s 89ms/step - accuracy: 0.8724 - loss: 0.3061 - val_accuracy: 0.8494 - val_loss: 0.3412
Epoch 3/5
391/391  30s 77ms/step - accuracy: 0.8900 - loss: 0.2673 - val_accuracy: 0.8456 - val_loss: 0.3569
Epoch 4/5
391/391  31s 79ms/step - accuracy: 0.9075 - loss: 0.2327 - val_accuracy: 0.8403 - val_loss: 0.3997
Epoch 5/5
391/391  33s 84ms/step - accuracy: 0.9224 - loss: 0.1996 - val_accuracy: 0.8372 - val_loss: 0.3838
Test loss: 0.3838488757610321
Test accuracy: 0.8371999859809875
```

Experiment 6: Named Entity Recognition using CRF (Conditional Random Fields)

```
# 6
import numpy as np
import pycrfsuite
import xml.etree.ElementTree as ET
from sklearn.metrics import classification_report
from sklearn.model_selection import train_test_split

# -----
# Parse XML
# -----
def add_tokens(texts, text, label):
    if text and text.strip():
        for word in text.split():
            texts.append((word, label))

def parse_xml(file_path):
    tree = ET.parse(file_path)
    root = tree.getroot()

    for elem in root.findall("./document"):
        texts = []

        text_tag = elem.find("textwithnamedentities")
        if text_tag is None:
            continue # skip invalid entries

        add_tokens(texts, text_tag.text, "I")
        for child in text_tag:
            label = "N" if child.tag == "namedentityintext" else "I"
            add_tokens(texts, child.text, label)
            add_tokens(texts, child.tail, "I")

        doc = np.array(texts, dtype=object)
        yield doc

# -----
# Feature Extraction
# -----
def word2features(doc, i):
    word = doc[i][0]

    features = [
        'bias',
```

```

        f'word.lower={word.lower()}',
        f'word[-3:]={word[-3:]}',
        f'word[-2:]={word[-2:]}',
        f'word.isupper={word.isupper()}',
        f'word.istitle={word.istitle()}',
        f'word.isdigit={word.isdigit()}',
    ]

    if i > 0:
        word1 = doc[i-1][0]
        features.append(f'-1:word.lower={word1.lower()}')
    else:
        features.append('BOS')

    if i < len(doc) - 1:
        word1 = doc[i+1][0]
        features.append(f'+1:word.lower={word1.lower()}')
    else:
        features.append('EOS')

    return features

def extract_features(doc):
    return [word2features(doc, i) for i in range(len(doc))]

def get_labels(doc):
    return [label for (_, label) in doc]

# -----
# Main
# -----
if __name__ == "__main__":

    file_path = "reuters.xml"

    docs = list(parse_xml(file_path))
    print("Sample doc:", docs[0])

    # Prepare data
    X = [extract_features(doc) for doc in docs]
    y = [get_labels(doc) for doc in docs]

    # Train-test split
    X_train, X_test, y_train, y_test, docs_train, docs_test =
train_test_split(
    X, y, docs, test_size=0.2, random_state=42

```



```

)

# Train CRF
trainer = pycrfsuite.Trainer(verbose=False)
for xseq, yseq in zip(X_train, y_train):
    trainer.append(xseq, yseq)

trainer.set_params({
    'c1': 0.1,
    'c2': 0.01,
    'max_iterations': 100,
    'feature.possible_transitions': True
})

trainer.train('crf.model')

# Predict
tagger = pycrfsuite.Tagger()
tagger.open('crf.model')

y_pred = [tagger.tag(xseq) for xseq in X_test]

i = 0

print("\nSample Prediction:")
for token, pred in zip([w[0] for w in docs_test[i]], y_pred[i]):
    print(f"{token} -> {pred}")

# Evaluation
labels = list(set(label for doc in y_train for label in doc))

y_true_flat = [label for seq in y_test for label in seq]
y_pred_flat = [label for seq in y_pred for label in seq]

print("\nClassification Report:")
print(classification_report(y_true_flat, y_pred_flat, zero_division=0))

```

OUTPUT:

```
Sample doc: [['This' 'I']  
             ['is' 'I']  
             ['a' 'I']  
             ['news' 'I']  
             ['about' 'I']  
             ['India' 'N']  
             ['. ' 'I']]
```

Sample Prediction:

Virat -> I

Kohli -> I

played -> I

well. -> I

Classification Report:

	precision	recall	f1-score	support
I	0.50	1.00	0.67	2
N	0.00	0.00	0.00	2
accuracy			0.50	4
macro avg	0.25	0.50	0.33	4
weighted avg	0.25	0.50	0.33	4

Experiment 7: Autoencoder for Image Reconstruction (MNIST)

```
# 7
# Import necessary libraries
from tensorflow.keras.layers import Input, Dense
from tensorflow.keras.models import Model
from tensorflow.keras.datasets import mnist
from tensorflow.keras.callbacks import TensorBoard

# Load MNIST dataset
(x_train, _), _ = mnist.load_data() # We only need the training set for now

# Define input shape and size
input_dim = 784 # For MNIST dataset
input_shape = (input_dim,)

# Reshape the data to fit the model
x_train = x_train.reshape(x_train.shape[0], input_dim).astype('float32') /
255.

# Define the encoder architecture
encoding_dim = 32
input_layer = Input(shape=input_shape)
encoder_layer1 = Dense(128, activation='relu', name='encoder_1')(input_layer)
encoder_layer2 = Dense(64, activation='relu',
name='encoder_2')(encoder_layer1)
encoder_layer3 = Dense(encoding_dim, activation='relu',
name='encoder_output')(encoder_layer2)
encoder_model = Model(inputs=input_layer, outputs=encoder_layer3)

# Define the decoder architecture with mirrored layers
encoded_input = Input(shape=(encoding_dim,))
decoded_layer1 = Dense(64, activation='relu',
name='decoder_1')(encoded_input)
decoded_layer2 = Dense(128, activation='relu',
name='decoder_2')(decoded_layer1)
decoded_layer3 = Dense(input_dim, activation='sigmoid',
name='decoder_output')(decoded_layer2)
decoder_model = Model(inputs=encoded_input, outputs=decoded_layer3)

# Define the full autoencoder model including both encoder and decoder
autoencoder_model = Model(inputs=input_layer,
outputs=decoder_model(encoder_model(input_layer)))

# Compile the model with optimization functions and an evaluation metric
autoencoder_model.compile(optimizer='adam', loss='binary_crossentropy',
metrics=['mse'])

# Set up TensorBoard only if it is installed
```

```

callbacks = []
try:
    from tensorflow.keras.callbacks import TensorBoard
    import tensorboard

    callbacks.append(TensorBoard(log_dir='./logs', histogram_freq=0))
except ImportError:
    print("TensorBoard is not installed. Training will continue without
TensorBoard logging.")

# Fit the autoencoder to the data
autoencoder_model.fit(x_train, x_train,
                      epochs=20,
                      batch_size=128,
                      shuffle=True,
                      validation_split=0.1,
                      callbacks=callbacks)

```

OUTPUT:

```

... Epoch 1/20
422/422 ————— 5s 8ms/step - loss: 0.2096 - mse: 0.0493 - val_loss: 0.1499 - val_mse: 0.0290
Epoch 2/20
422/422 ————— 3s 8ms/step - loss: 0.1361 - mse: 0.0245 - val_loss: 0.1259 - val_mse: 0.0209
Epoch 3/20
422/422 ————— 3s 8ms/step - loss: 0.1215 - mse: 0.0196 - val_loss: 0.1177 - val_mse: 0.0181
Epoch 4/20
422/422 ————— 3s 8ms/step - loss: 0.1137 - mse: 0.0170 - val_loss: 0.1109 - val_mse: 0.0159
Epoch 5/20
422/422 ————— 3s 8ms/step - loss: 0.1088 - mse: 0.0154 - val_loss: 0.1079 - val_mse: 0.0149
Epoch 6/20
422/422 ————— 3s 8ms/step - loss: 0.1056 - mse: 0.0143 - val_loss: 0.1045 - val_mse: 0.0138
Epoch 7/20
422/422 ————— 3s 8ms/step - loss: 0.1031 - mse: 0.0135 - val_loss: 0.1023 - val_mse: 0.0131
Epoch 8/20
422/422 ————— 3s 8ms/step - loss: 0.1009 - mse: 0.0128 - val_loss: 0.1006 - val_mse: 0.0125
Epoch 9/20
422/422 ————— 3s 8ms/step - loss: 0.0990 - mse: 0.0122 - val_loss: 0.0987 - val_mse: 0.0120
Epoch 10/20
422/422 ————— 3s 8ms/step - loss: 0.0974 - mse: 0.0117 - val_loss: 0.0976 - val_mse: 0.0116
Epoch 11/20
422/422 ————— 3s 6ms/step - loss: 0.0961 - mse: 0.0112 - val_loss: 0.0958 - val_mse: 0.0110
Epoch 12/20
422/422 ————— 2s 4ms/step - loss: 0.0950 - mse: 0.0109 - val_loss: 0.0950 - val_mse: 0.0108
Epoch 13/20
...
Epoch 19/20
422/422 ————— 3s 8ms/step - loss: 0.0911 - mse: 0.0097 - val_loss: 0.0918 - val_mse: 0.0097
Epoch 20/20
422/422 ————— 3s 8ms/step - loss: 0.0906 - mse: 0.0095 - val_loss: 0.0911 - val_mse: 0.0095
Output is truncated. View as a scrollable element or open in a text editor. Adjust cell output settings...
... <keras.src.callbacks.history.History at 0x1f05a944c90>

```